# Maze: A Distributed Framework for Large Language Model Agents

Jing Gu[1], Zhuang Xing[1], Yiheng Yang[1], Bowen Lv[1], Jiale Wang[1], Shuo Yuan[2], Zijin Chen[1], Jin Zhao[1], Pengfei Zuo[3], Long Zheng[1], Xiaofei Liao[1], Hai Jin[1], Qinbin Li[1*]

[1] National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, Hubei, China

[2] Nanjing University of Aeronautics and Astronautics, Nanjing, China

[3] Individual Researcher, Beijing, China

***Abstract*—LLM agents are transitioning from single-node to large-scale, multi-tenant services. A user request no longer corresponds to a single inference call, but to a long-running workflow interleaving heterogeneous tasks. Existing agent systems struggle with these workloads due to resource-blind, coarse-grained placement or a lack of agentic workload awareness.**

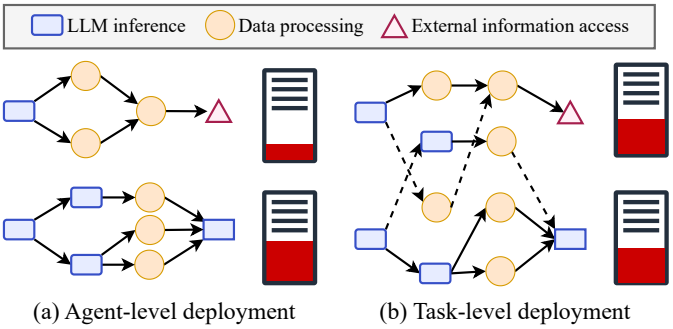
**We present Maze, a distributed agent framework bridging the abstraction gap from agentic logic to fine-grained task-level deployment. At its core, Maze anchors task resource semantics and introduces Heterogeneity-Aware Criticality Scheduler, a scheduler leveraging agentic workload features and value-density scoring to prioritize valuable agentic workflows and critical tasks while eliminating cross-resource head-of-line blocking. Coupled with elastic inference engine scaling, zero-VRAM optimizations, and closed-loop fault tolerance, Maze ensures a robust, performant runtime. Evaluation on diverse workloads demonstrates that Maze achieves up to 2.9× higher throughput, reduces P95 latency by 54.2%, and increases GPU utilization by 50% compared to state-of-the-art frameworks.**

***Index Terms*—LLM agent, distributed systems**

## I. Introduction

The rapid proliferation of Large Language Model (LLM) agents [1], [2] is reshaping how AI services are deployed. Instead of isolated single-node prototypes, modern AI applications increasingly operate as production-grade, multi-tenant cloud services. In this emerging paradigm, a single user request rarely corresponds to one model invocation. Instead, it typically triggers a long-running workflow involving tool calls, database queries, LLM or VLM inference, and various post-processing steps. Whether predefined by developers or generated by an LLM planner, these workflows manifest as Directed Acyclic Graphs (DAGs) that interleave diverse operations: model inference, data processing, and tool execution. Providers must schedule these concurrent, heterogeneous workflows on shared clusters to satisfy strict Service-Level Objectives (SLOs) while simultaneously maintaining high utilization of heterogeneous compute resources.

Our comprehensive measurement of representative agentic workloads across GAIA [3], $\tau$-bench [4], and OpenAGI [5] reveals that these workloads differ fundamentally from traditional inference or batch jobs. 1) *Extreme heterogeneity*: GPU-bound LLM/VLM inference, CPU-bound processing, and I/O-bound tool calls are interleaved within the same DAG. 2) *Strong dependencies*: intermediate results flow along DAG edges and shape subsequent prompts and tool invocations. 3) *Inference intensive*: due to the high cost of LLM inferences, they often dominate end-to-end latency in many workflows.




(a) Agent-level deployment (b) Task-level deployment

Fig. 1: Comparison of agent-level and task-level deployment. Agent-level heterogeneity leads to load imbalance across machines under agent-level deployment. Fine-grained task-level deployment achieves a better balance.

Unfortunately, existing systems suffer from an abstraction mismatch when handling agentic workloads. Agent frameworks such as LangChain [6] and AutoGen [7] focus on programming convenience and deploy at agent granularity: each agent is pinned to a compute node (Figure 1a), which hides internal structure and resource heterogeneity from the cluster. This results in severe load imbalance, low GPU utilization, and high tail latency. In contrast, model-serving systems [8], [9] such as Agentix excel at scheduling individual model inference calls on GPUs but remain oblivious to the surrounding CPU and I/O tasks. Thus, they do not consider optimizations from the perspective of agentic workflow. General-purpose task schedulers like Ray [10] and even recent agent-specific orchestrators such as Ayo [11] not only remain resource-blind at the task level but also lack specialized scheduling and

*Corresponding author: Qinbin Li (qinbin@hust.edu.cn).

SC26, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-4789-7/26/$31.00 ©2026 IEEE

agentic runtime optimizations.

These observations suggest that we must *shift the abstraction boundary* when designing frameworks for LLM agents. Instead of treating an agent as a black box that owns a process or a model endpoint, we argue that an agent should be represented as a workflow of tasks and executed at the task level by the runtime, as shown in Figure 1b. By exposing this internal structure, the system can achieve fine-grained resource anchoring for precise task placement and perform agent workload-aware scheduling. However, turning this idea into a practical system raises three challenges.

**Challenge 1: Bridging the abstraction gap from agentic logic to task-level deployment.** We lack a unified abstraction that anchors fine-grained resources to tasks, which is essential to unlock the potential of distributed compute clusters. Existing engines like Ray [10] support numerical resource requests but lack the semantics to automatically map agentic logic into resource-anchored tasks. This abstraction gap compels developers to manually estimate task overhead and forces schedulers to assign tasks using coarse-grained resource quotas. Consequently, even recent agentic orchestrators like Ayo [11] struggle to maximize hardware efficiency in heterogeneous distributed environments.

**Challenge 2: Scheduling dependency-rich and heterogeneous workflows.** Agentic workloads are characterized by their resource heterogeneity (interleaving CPU/GPU/I/O), complex task dependencies, and intensive inference. Consequently, conventional schedulers fail to capture these interdependent signals, causing cross-resource head-of-line (HoL) blocking and prioritizing task granularity over end-to-end workflow progression. Specifically, agent-level frameworks incur severe resource locking via coarse-grained execution, while traditional schedulers are ineffective because they ignore the heterogeneity and inference intensity of agentic workloads.

**Challenge 3: Building a scalable and practical runtime.** A task-level abstraction requires a scalable runtime that can execute a large number of fine-grained tasks across a cluster without overwhelming the control plane. Specifically, the runtime must ensure robustness through agent-specific fault tolerance, while maximizing end-to-end efficiency through specialized context management and elastic scheduling of LLM serving stacks. Although general-purpose distributed engines (e.g., Ray and Kubernetes [12]) provide essential building blocks, they overlook the critical design nuances required to construct a reliable and performant runtime specifically optimized for the complex workloads of LLM agents.

To address these challenges, we present Maze, a distributed task-oriented framework for serving LLM agents at scale. First, to bridge the abstraction gap, Maze shifts the deployment paradigm from coarse-grained agents to fine-grained heterogeneous tasks. Through a high-level Python API, Maze compiles agent logic into an executable DAG of typed tasks with explicit resource anchoring. This allows the system to perform precise resource estimation and decouples developers from the complexities of distributed task placement. Second, to handle the scheduling heterogeneity and dependencies, Maze introduces a heterogeneity-aware criticality scheduler using online learning to predict task runtimes from input features, and prioritizes tasks based on a generalized value-density score. It dispatches tasks through separate GPU/CPU/I/O queues to eliminate cross-resource head-of-line blocking. Third, to ensure a robust and performant runtime, Maze incorporates a suite of agent-native mechanisms. It features adaptive inference engine scheduling that enables elastic scaling across LLM serving engines, complemented by DAG-aware context management for maximizing KV cache reuse and a zero-VRAM standby mechanism for low-latency startup. Crucially, Maze implements a closed-loop fault tolerance strategy that recovers from resource-driven failures (e.g., OOM) by dynamically re-anchoring task demands, ensuring operational stability.

We have implemented Maze on top of Ray [10] with 27K lines of code (excluding Ray's code). We evaluate it on a suite of agent workloads constructed from GAIA [3], $\tau$-bench [4], and OpenAGI [5]. Results demonstrate that Maze outperforms agent-level, model-level, and task-level methods, improving throughput by up to 2.9×, reducing P95 response time by 54.2%, and achieving 50% higher GPU utilization.

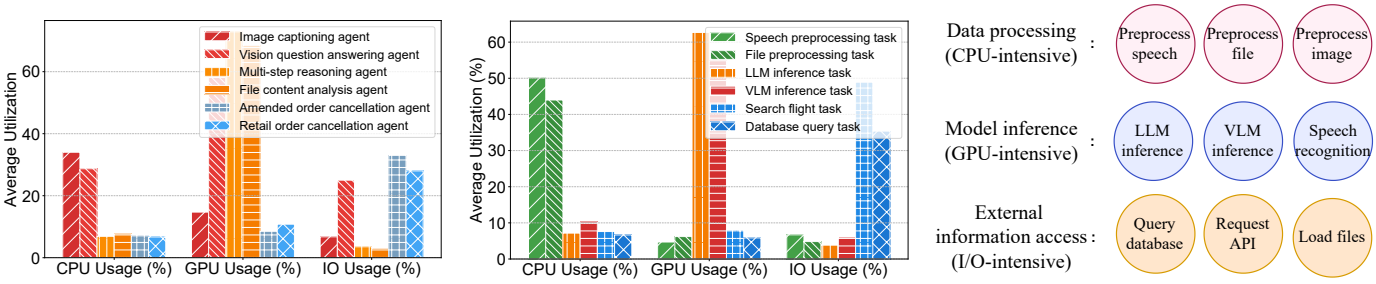
In summary, this paper makes the following contributions:

- We characterize realistic LLM agent workloads as heterogeneous, dependency-rich, inference-intensive DAGs, and show that existing agent-level, model-serving and task-level systems cannot handle them effectively, motivating a fine-grained task-oriented execution paradigm.
- We design Maze, a distributed framework that bridges the abstraction gap from agentic logic to task-level deployment. It integrates resource-anchored task transformation, specialized scheduling, and an agent-native runtime with elastic scaling and optimizations to provide a robust and efficient serving environment.
- We propose HACS, a heterogeneity-aware criticality scheduler that combines online task-time prediction, generalized value-density scoring, and heterogeneous resource queues to jointly optimize throughput and tail latency under multi-tenant loads.
- We evaluate Maze on a real cluster and demonstrate that it outperforms state-of-the-art agent-level, model-level, and task-level systems across a range of realistic workloads, while scaling effectively with cluster size.

## II. Background and Motivation

### A. LLM Agents as Workflows

LLM agents extend foundation models by integrating reasoning with planning, memory, and tool-use modules [1]. These components enable LLM agents to autonomously handle complex tasks such as answering open-ended questions, automating software debugging, or conducting iterative data analysis [13]–[15]. A widely adopted and practical paradigm for LLM agents is the *workflow-based* design [16], [17], where control flows are represented as Directed Acyclic Graphs (DAGs) of interdependent tasks. In this paper, we focus on this workflow view rather than purely step-by-step




(a) Resource consumption across agents (b) Resource consumption across tasks (c) Examples of types of tasks

Fig. 2: Observations of resource usage across agents and tasks

generation where the agent creates only a single next node after each completion, as the latter incurs substantial overhead and leads to unpredictable latency in large-scale deployments. Our framework also accommodates incremental workflows, where the planner generates the next few steps as a sub-DAG at a time; these sub-DAGs are stitched into the global workflow and executed under the same task-level abstraction and scheduling mechanisms. Serving LLM agents with long-running, heterogeneous workflows poses a critical challenge: providers must arbitrate these dynamic and diverse resource demands to enforce strict Service-Level Objectives (SLOs) while maximizing cluster utilization.

### *B. Characterizing Agentic Workloads*

To understand the requirements of LLM agent serving, we profiled 14 representative agents from GAIA [3], $\tau$-bench [4], and OpenAGI [5]. Our analysis reveals three properties that distinguish agentic workloads from traditional jobs.

**Taxonomy of Agent Tasks.** We observe that agent tasks naturally fall into three distinct resource categories (Figure 2c).

- **Model inference (GPU-intensive):** Agents frequently perform multiple model inferences for understanding, planning, and reasoning. These include large-scale LLM/VLM forward passes as well as smaller model inferences within specialized tools. Such tasks are highly parallelizable and place heavy demand on GPUs.
- **Data processing (CPU-intensive):** Agents often require data preprocessing or postprocessing to align inputs and outputs with downstream tasks. These operations, such as text parsing, feature extraction, or result formatting, are dominated by sequential CPU computation.
- **External information access (I/O-intensive):** To enrich context, agents commonly retrieve information from external sources, such as reading local files, or querying databases. The performance of these tasks is constrained by network and disk I/O latency.

This taxonomy reveals three key workload properties:

- **Drastic Heterogeneity.** Unlike homogeneous workloads, an agent DAG interleaves these three task types (Figure 2a). A single workflow may jump from a GPU inference to a CPU parse, then to an I/O search, triggering complex cross-resource contention (Figure 2b).

TABLE I: Comparison of popular LLM agent frameworks on distributed capabilities

| Framework | Distributed Deployment | Resource-Aware | Scheduling | Load Balancing |
|---|---|---|---|---|
| LangChain | ✗ | ✗ | ✗ | ✗ |
| AgentScope | Agent-level (gRPC) | ✗ | ✗ | ✗ |
| CrewAI | ✗ | ✗ | ✗ | ✗ |
| AutoGen | Agent-level (gRPC) | ✗ | ✗ | ✗ |
| Phidata | ✗ | ✗ | ✗ | ✗ |
| PydanticAI | ✗ | ✗ | ✗ | ✗ |
| **Maze** | **Task-level** | ✓ | ✓ | ✓ |

- **Strong Dependencies.** Tasks are strictly ordered by data flow. The output of a tool call becomes the input prompt for the next inference. This dependency restricts parallelism and makes the end-to-end latency dictated by the critical path [9].
- **Inference Intensity.** Agent workflows are characterized by significant *inference intensity*, as exemplified by complex patterns like LLM-powered search [18], decision-making [19] and code-generation evaluation [20]. In these scenarios, the cumulative execution time of GPU-bound inference tasks constitutes a substantial portion of the end-to-end response latency.

### *C. Inefficiency of Existing Deployments*

We examine three prevailing deployment paradigms and demonstrate a fundamental abstraction mismatch between their design assumptions and the properties of agentic workloads.

**Agent-Level Deployment.** To support the growing ecosystem of LLM agents, various open-source frameworks have been developed, such as LangChain [6], AgentScope [21], CrewAI [22], AutoGen [7], Phidata [23], and PydanticAI [24]. These frameworks provide high-level abstractions for agent definition, tool integration, memory management, and chaining LLM calls. To support a distributed execution, frameworks such as AutoGen [7] and AgentScope [21] adopt a coarse-grained deployment strategy, treating each agent as an indivisible unit statically bound to a compute node. Table I summarizes the distributed capabilities of representative LLM agent frameworks. However, this deployment approach fails to capture the internal heterogeneity of agent workloads [25].

Statically pinning agents to nodes triggers severe resource locking, where instances reserve GPU memory even during CPU or I/O phases, thereby fragmenting utilization and preventing the cluster from interleaving heterogeneous multi-tenant requests.

**Model-Level Serving.** Systems like vLLM [8] and Agentix [9] operate fundamentally at the granularity of inference requests. This narrow scope causes the scheduler to blindly optimize GPU throughput while the workflow stalls on unmanaged non-LLM bottlenecks. Furthermore, without visibility into the workflow's critical path, these systems rely on fairness-based scheduling policies (e.g., PLAS/ATLAS [9]) to prevent starvation. Paradoxically, this pursuit of fairness deprioritizes critical tasks in static workflows, resulting in higher end-to-end latencies compared to dependency-aware scheduling.

**Task-level Deployment.** Distributed frameworks such as Ray [10] provide a robust substrate for fine-grained execution. However, Ray's native scheduler is workflow-agnostic; it schedules tasks independently as soon as dependencies are resolved, ignoring the inherent structure of agentic workflows and SLO requirements. While recent orchestrators like Ayo [11] introduce primitive-level parallel optimizations, they lack a resource-anchoring mechanism and do not offer specialized task-level scheduling or a stable runtime tailored to agentic workload characteristics. Consequently, these systems fail to bridge the critical abstraction gap between high-level agentic logic and efficient, fine-grained task deployment.

### D. Motivation and Challenges

The observations above motivate Maze, a fine-grained, task-oriented serving framework specifically designed for LLM agents. Maze bridges the gap from high-level agentic workflow execution to fine-grained task-level deployment by precisely perceiving resource semantics and employing a distributed scheduler and runtime natively tailored to agentic workload characteristics. By doing so, Maze provides an efficient, scalable, and robust infrastructure for agent serving.

However, building such a system presents three challenges. *1) The Agent Abstraction Gap:* We need an abstraction that allows developers to write agents naturally while automatically exposing fine-grained resource anchoring (e.g., GPU/CPU/I/O) to make informed placement decisions. Bridging this gap requires an abstraction that automatically extracts these semantics from high-level code without compromising programming convenience. *2) Heterogeneity-Aware Scheduling:* The scheduler must solve a dual problem: it must isolate heterogeneous resources to prevent HoL blocking, while simultaneously identifying and prioritizing critical-path tasks across these isolated queues to minimize end-to-end latency. *3) Scalable Runtime:* Decomposing agents into fine-grained tasks increases control plane pressure. Specifically, it must provide efficient context management and fast task deployment, while enabling elastic scaling of inference engines and closed-loop fault tolerance to ensure a highly performant and robust execution environment.

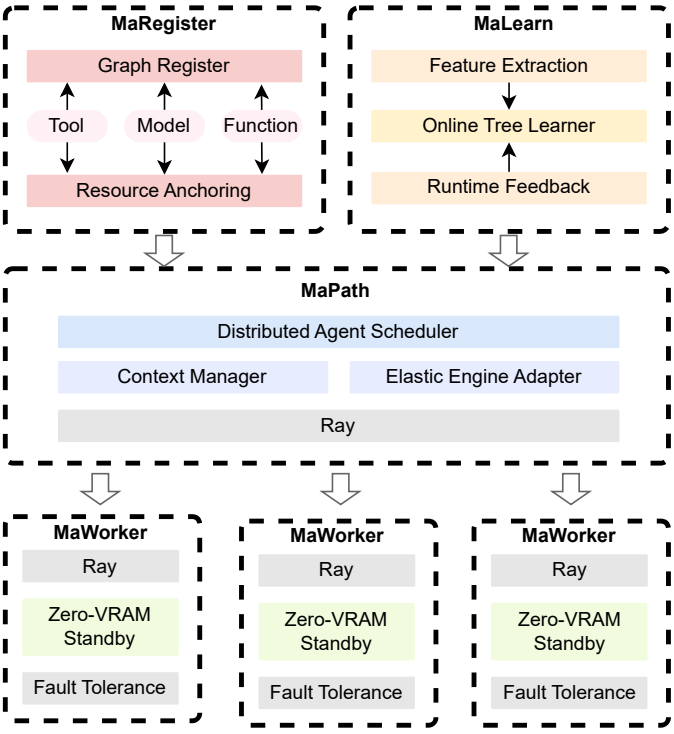



Fig. 3: The architecture overview of Maze

## III. The Maze Design

To support efficient large-scale LLM agent serving, we present Maze, a distributed task-oriented framework. This section first presents the system overview and the task-level abstraction of Maze, which bridges the gap between high-level agentic logic and resource-anchored tasks (§III-A). We then detail the heterogeneity-aware criticality scheduler, which leverages tree-based complexity predictors to perform workflow-centric optimization (§III-B). Finally, we describe the scalable and robust agent-native runtime designed for stable and high-performance execution (§III-C).

**Task Model.** Maze logically models each task as $\tau = (id, type, deps, resources, state)$, where $type \in \{\mathrm{CPU}, \mathrm{GPU}, \mathrm{I/O}\}$ denotes its resource category, $deps$ denotes its predecessor tasks in the workflow DAG, and $resources$ specifies its CPU, memory, and GPU requirements. A task with unsatisfied dependencies remains *pending*; once all predecessors complete, it becomes eligible and is inserted into the corresponding heterogeneous resource queue. It is then tracked as *queued* and becomes *running* when dispatched to a MaWorker, eventually reaching *succeeded*, *failed*, or *timed_out* under normal execution. The same task semantics and dependency model apply to dynamically appended tasks.

### A. System Overview and Task-level Abstraction

As shown in Figure 3, Maze integrates the **MaRegister** module to enforce resource-semantic abstractions, directly bridging the agent abstraction gap (Challenge 1).

- **MaRegister (Abstraction Interface):** allows developers to define agents while automatically extracting task-level

TABLE II: Maze's core APIs

| Name | Description |
|---|---|
| *agent* = maze.DAGAgent(*config*) | Initializes a distributed agent instance and returns its management handle based on the provided configuration. |
| *agent*.register_task(*func*) | Registers a computational function as a fine-grained task node within the agent's internal DAG. |
| *task_id* = *agent*.register_link(*func_name, inputs*) | Performs dependency resolution by parsing inputs to establish directed data-flow links to the specified task. |
| *exec_cont* = *agent*.submit(*mode*) | Submits workflow within the agent based on the `mode`, returning an execution context containing a unique `run_id`. |
| *result* = *agent*.get(*task_id, run_id*) | Retrieves the latest execution status and result of task in the workflow. |
| *ok* = *agent*.destroy(*run_id*) | Terminates the specified workflow execution and triggers cleanup of associated distributed resources. |

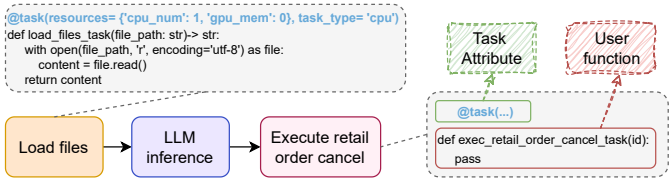



Fig. 4: DAG and node attributes

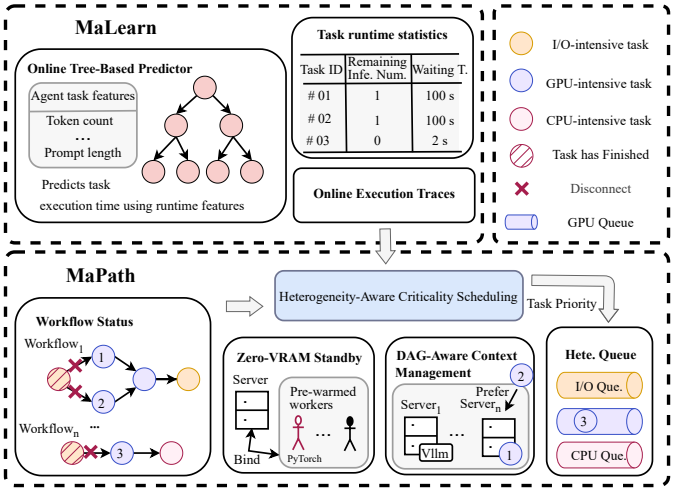



Fig. 5: Agent workflow scheduling with HACS

DAGs. It infers resource requirements (e.g., CPU cores, GPU memory) to enable fine-grained resource anchoring for each task node (Challenge 1).

- **MaLearn (Prediction):** maintains an *online lightweight tree-based model* that predicts the complexity of tasks.
- **MaPath (Scheduling):** implements *heterogeneity-aware criticality scheduling* (Challenge 2) by combining detailed resource semantics and predicted durations for task assignment. It also manages execution contexts and inference engines.
- **MaWorker (Execution Runtime):** provides a robust and scalable environment (Challenge 3). It features Zero-VRAM standby and closed-loop fault tolerance that monitors execution failures (e.g., OOM) to update task resource semantics and trigger rescheduling.

Unlike general-purpose systems like Ray that treat tasks as opaque compute units, Maze enforces a **resource-semantic abstraction** via MaRegister (Figure 4). For any Python function, even if it is not explicitly marked with `@task`, Maze automatically performs a two-phase resource anchoring. *1) Coarse-Grained Static Anchoring:* MaRegister first performs lightweight static program analysis over imports/API calls, model specifications, and task annotations to infer initial task types and baseline resource requirements. For instance, detecting a task invoking a Qwen-32B model in BF16 format allows Maze to estimate a mandatory footprint of ∼70GB VRAM (based on model parameters and precision). This initial estimate classifies tasks into three categories (GPU, CPU, or I/O-intensive) to facilitate subsequent fine-grained task-level assignment. *2) Fine-Grained Dynamic Anchoring:* At runtime, Maze monitors actual consumption and maps task inputs to resource costs, building an empirical input-to-resource view. This feedback enables fine-grained anchoring for subsequent executions of similar tasks. If an Out-of-Memory error occurs, the MaWorker fault tolerance mechanism captures the failure, updates the task's resource semantics based on the actual resource overhead and error information, and triggers a rescheduling of the task.

**Workflow Lifecycle.** As summarized in Table II, the execution of an agent request follows a streamlined lifecycle. In Maze, agentic logic is represented as a dependency-preserving workflow of steps, such as LLM inference, tool invocations, and data processing. Developers define this logic by registering tasks via register_task and establishing dependencies through register_link. Upon submit(), MaRegister compiles the definitions into a resource-anchored DAG, estimating baseline demands to guide initial placement. As predecessor tasks complete, MaLearn intercepts the inputs of ready tasks to dynamically predict their execution durations. As illustrated in Figure 5, MaPath combines these temporal signals with the visible DAG topology to calculate task priorities and perform fine-grained assignment to appropriate MaWorkers. The selected MaWorkers then execute the assigned tasks. Finally, users retrieve execution results via get(), and the system invokes destroy() to release distributed resources and clean up the DAG context.

### B. Heterogeneity-Aware Criticality Scheduling via Online Learning

Building on the fine-grained resource semantics from MaRegister, Maze leverages MaLearn to extract predictive signals and MaPath to perform heterogeneity-aware scheduling (Challenge 2).

**Predictive Signals via Online Learning.** While agent workflows are implemented via general-purpose code, their runtime characteristics are far from random. Our analysis reveals that execution times for recurring patterns (e.g., LLM inference, speech processing, and I/O) correlate strongly with measurable attributes like token counts, audio duration, or file sizes (Pearson correlation $> 0.7$). Such predictability provides the scheduler with valuable signals to optimize resource allocation based on estimated duration. Motivated by the above observation, Maze integrates MaLearn, a module that collects task execution traces and supports execution time prediction to inform scheduling decisions. It employs Gradient Boosted Decision Trees (GBDTs) [26], [27] to map task features to runtimes while maintaining a low computational overhead. Instead of relying on pre-trained models, MaLearn incrementally builds its predictive capability by collecting samples during system operation, using features extracted from predecessor task outputs to predict successor runtimes. To ensure efficiency, MaLearn performs model training in the background. As established in Theorem 1, its update mechanism (formally a sliding-window *prune–add* strategy) ensures a stable loss descent by replacing stale trees with those better aligned with current gradients, allowing Maze to adapt to distribution shifts without performance regression. Upon a task's completion, Maze identifies successors that necessitate duration estimates and triggers the prediction only after verifying that all required input features have been successfully retrieved.

**Theorem 1** (Convergence)**.** *For a smooth loss $\mathcal{L}$, small $\eta > 0$, and new tree $f^*$ more aligned with $-\nabla\mathcal{L}$ than the pruned $f_{old}$ (i.e., $\langle -\nabla\mathcal{L}, f^* - f_{old}\rangle > 0$), the first-order net loss change satisfies: $\Delta_{total} \approx \eta\langle\nabla\mathcal{L}, f^* - f_{old}\rangle < 0$.*

**Heterogeneity-Aware Criticality Scheduling.** The scheduling algorithm provides the temporal execution logic for the distributed scheduler, designed around the key properties of agent workloads: inference-heavy DAG structures and heterogeneous tasks. HACS adapts and enhances the classic $c\mu$ rule [28], redefining the value density of tasks in agent workflows through the lenses of execution efficiency and system fairness.

By leveraging execution time predictions provided by MaLearn, HACS calculates a unit-value priority score. For static workflows, the score is defined as follows:

$$S(\tau) = \frac{\Omega(\tau)\beta^{\Phi(\tau)}}{T_{pred}(\tau)}$$

where $\Phi(\tau) = \frac{\tau_{wait}}{\alpha \cdot \bar{dct}} - N_{val}$ represents the Workflow Value term, in which $\tau_{wait}$ is the elapsed time (in seconds) since the entire workflow was submitted, $\bar{dct}$ is the exponential moving average (EMA) of the global DAG completion time, $\alpha$ is the starvation coefficient, and $N_{val}$ denotes the number of value-critical tasks remaining in the workflow. Since inference operations are compute-intensive components in agentic workloads, we define $N_{val}$ as the count of remaining inference tasks by default, though users may customize this definition for specific scenarios (e.g., RAG). Within this framework, an exponential base $\beta$ is employed to significantly amplify the priority of workflows as they approach completion, while the term $\frac{\tau_{wait}}{\alpha \cdot \bar{dct}}$ ensures the priority escalates over time to prevent starvation. Additionally, $\Omega(\tau) = \log(2 + 2N_{desc})$ represents the Topological Weight that reflects task criticality, where $N_{desc}$ is the longest topological distance from the current task $\tau$ to the end node. This ensures that tasks bottlenecking the longest critical paths are prioritized to accelerate workflow completion. Finally, $T_{pred}(\tau)$ is the predicted execution duration of the task provided by MaLearn.

In dynamic workflows where the global topology is non-deterministic (e.g., involving self-reflection or dynamic branching), Maze assumes that nodes situated further from the start node are more likely to reside on the critical path. To accommodate this uncertainty, we set $\Phi(\tau) = 0$ and redefine $\Omega(\tau) = N_{anc} + \frac{\tau_{wait}}{\alpha \cdot \bar{dct}}$, where $N_{anc}$ represents the topological distance from the workflow entry to the current node $\tau$. To prevent cross-resource head-of-line blocking, Maze dispatches ready tasks into three isolated heterogeneous queues (GPU, CPU, and I/O) according to their resource requirements and orders tasks within each queue by the priority score $S(\tau)$.

**DAG-Aware Context Management.** To complement temporal scheduling, Maze employs a DAG-aware placement strategy that optimizes both data access and inference latency. First, Maze maintains a resident DAGContext for each workflow, anchored to an Affinity Node. Tasks are preferentially routed to their Affinity Node, so they can access shared state (e.g., intermediate files) locally. When the Affinity Node is overloaded, the scheduler falls back to load balancing by dispatching tasks to the least-loaded node, preventing hot spots while preserving locality when possible. Second, for GPU-bound inference tasks, Maze implements a KV-cache–aware routing policy: subsequent requests for the same model within a workflow are routed to the same model service instance, improving KV-cache hit rate and reducing prefill overheads.

**Demand-Driven Inference Scaling.** Maze supports mainstream inference engines and manages their lifecycle through a demand-driven strategy. When the ratio of pending requests for a specific model to its active instances exceeds an empirical threshold, the scheduler asynchronously triggers the deployment of engine instances on idle GPUs. Conversely, to reclaim expensive VRAM resources, the system automatically evicts instances that remain idle based on a Least Recently Used (LRU) policy. This elastic provisioning allows Maze to dynamically adapt to fluctuating agent workloads, ensuring responsiveness while maximizing hardware utilization.

### C. Scalable and Robust Agent-Native Runtime

Building on the scheduling decisions, Maze implements a scalable and robust runtime that reduces execution latency via lookahead provisioning, enhances system stability through

TABLE III: Complete workflow catalog used in our evaluation. → denotes task dependency and ‖ denotes parallel branches.

| Benchmark | Workflow | Workflow Structure |
|---|---|---|
| OpenAGI | Document question answering | Load files → (Extract text content ‖ LLM inference) → Merge document analysis → Batch files → (LLM-a ‖ LLM-b ‖ LLM-c) → Merge results → Write results |
| OpenAGI | Image captioning | Load files → (Extract BLIP captions ‖ Extract OCR text) → Batch files → (VLM-a ‖ VLM-b ‖ VLM-c ‖ VLM-d) → Merge results → Write results |
| OpenAGI | Vision question answering | Load files → Batch files → (VLM-a ‖ VLM-b ‖ VLM-c) → Merge results → Write results |
| OpenAGI | Multilingual text analysis | Load files → Language detect → Translate text → (Text summarization ‖ Sentiment analysis) → Batch files → (LLM-a ‖ LLM-b ‖ LLM-c) → Merge results → Write results |
| $\tau$-bench | Airline booking transaction | Load files → LLM inference → (Search direct flights ‖ Search one-stop flights ‖ Get user details) → LLM inference → Book reservation |
| $\tau$-bench | Airline cancellation transaction | Load files → LLM inference → Get user and reservation details → Cancel airline order → Search new flights → LLM inference → Book new reservation |
| $\tau$-bench | Retail order cancellation | Load files → LLM inference → Cancel retail order |
| $\tau$-bench | Amended order cancellation | Load files → LLM inference → (Find user ‖ Get order details) → Execute operations |
| $\tau$-bench | Retail order modification | Load files → LLM inference → (Find user ‖ Get order details) → Execute modifications |
| $\tau$-bench | Retail order return processing | Load files → LLM inference → Find user → Get order details → Execute return |
| GAIA | File content analysis | Load files → Preprocess file → (LLM-a ‖ LLM-b) → Fuse LLM answers → Write results |
| GAIA | Multi-step reasoning | Load questions → (LLM-a ‖ LLM-b) → Fuse LLM answers → Write results |
| GAIA | Speech processing | Load files → Preprocess speech → Speech recognition → (LLM-a ‖ LLM-b) → Fuse LLM answers → Write results |
| GAIA | Vision question answering | Load files → Preprocess image → VLM inference → Write results |

application-level fault tolerance, and maintains seamless compatibility with the existing agent ecosystem.

**Zero-VRAM Standby via Lookahead Provisioning.** To mitigate worker startup and library import overheads (e.g., PyTorch), Maze maintains a standby pool of pre-warmed heterogeneous workers on each node. This mechanism operates in two stages: *1) VRAM-Free Standby Pool*, During pool initialization, workers are categorized into three types (GPU, CPU, and IO) to optimize resource pre-allocation. GPU workers are initialized with zero GPU resource allocation to avoid premature locking, allowing them to preload heavy libraries exclusively into host memory. Conversely, non-GPU workers are initialized with only lightweight dependencies, primarily to amortize process startup overheads. *2) Just-in-Time (JIT) Binding*, Each compute node maintains a local pool of these pre-warmed heterogeneous workers. When Maze dispatches a GPU task, it assigns a specific GPU index to a standby worker and performs JIT Binding by dynamically injecting the CUDA_VISIBLE_DEVICES environment variable immediately before execution. This warm execution mechanism eliminates both process startup and library loading overheads, reducing task startup latency from seconds to milliseconds.

**Application-Level Fault Tolerance.** While Ray provides infrastructure-level recovery (e.g., node failures via lineage re-execution), Maze focuses on mitigating agent-specific failures through two mechanisms: *1) Invocation Correction*, If an information extraction failure occurs (e.g., a JSON parsing error) due to improper max_token configuration, the executor automatically resets the parameter to its maximum allowed value and retries the task once. This ensures data flow integrity against LLM configuration errors. *2) Resource-Adaptive Retry*, Upon detecting a GPU Out-of-Memory error, the scheduler avoids immediate workflow termination. Instead, it leverages MaRegister to re-detect and re-allocate appropriate resources before re-executing the task once. This dynamically corrects imprecise task resource estimations and prevents interruptions from recoverable resource issues.

**Ecosystem Compatibility.** Maze provides adapters for frameworks like LangChain that intercept native chain executions and transparently transform them into Maze's task-level intermediate representation. This endows legacy agents with distributed execution capabilities to realize efficient scheduling and load balancing by leveraging the maze library.

## IV. Implementation

We have implemented Maze with approximately 27K lines of code using Python, JavaScript, and TypeScript, building the core distributed logic atop the Ray framework.

**State and DAGContext.** To manage data between tasks, Maze assigns each workflow a dedicated Ray Actor (DAGContext) and a unique namespace for isolation. We decouple task status signals, which are synchronized via Redis, from task outputs, which are managed by the DAGContext utilizing the Ray Plasma Object Store. The DAGContextManager maintains these states and only triggers resource reclamation upon receiving an explicit workflow termination signal.

**Online Prediction via MaLearn.** Maze uses MaLearn to map domain-specific features (e.g., token counts, audio duration) to task runtimes via XGBoost and LightGBM. By employing online learning and incremental training, it provides runtime estimates with negligible prediction overhead.

## V. Evaluation

We evaluate Maze's performance and architectural advantages through experiments on a multi-node distributed cluster. Our analysis spans two dimensions: *1) System-level Comparison*, comparing Maze against state-of-the-art agent-level, model-level, and task-level baselines to demonstrate end-to-end performance gains; *2) Algorithmic Efficacy*, isolating the impact of HACS by comparing it against alternative scheduling policies within the Maze runtime.

### A. Experimental Setup

**Workloads.** Our evaluation is based on three popular LLM agent benchmarks: GAIA [3], $\tau$-bench [4], and OpenAGI [5]. We select 14 representative workflow archetypes from these benchmarks, including document question answering, vision question answering, airline and retail transactions, multi-step reasoning, and speech processing, which collectively exhibit diverse CPU/GPU/I/O compositions. Table III lists all 14 workflows and their complete DAG structures in compact notation. We map each benchmark query to its corresponding workflow and construct a query pool of 1,876 problems. All subsequent experiments sample from this query pool.

**Testbed and Models.** Our primary experiments are conducted on a 32-node cluster running Ubuntu 22.04. Each node features an Intel Xeon Platinum 8358P CPU (15 cores), 250 GB RAM, and an NVIDIA A100 (80 GB) GPU. Nodes are interconnected via a 2 Gb/s LAN. To handle heterogeneous agent tasks, we deploy three 32B-parameter models: Qwen3, DeepSeek-R1, and Qwen2.5-VL. Unless specified otherwise, we utilize 8 compute nodes for default evaluation. For the cross-architecture evaluation, we additionally use an 8-NPU Ascend server running openEuler 22.03 LTS, equipped with four Kunpeng-920 CPUs (192 cores), 2 TB RAM, and eight Ascend 910B3 NPUs with 64 GB HBM each. On this platform, we deploy Qwen3-4B and Qwen2.5-VL-3B.

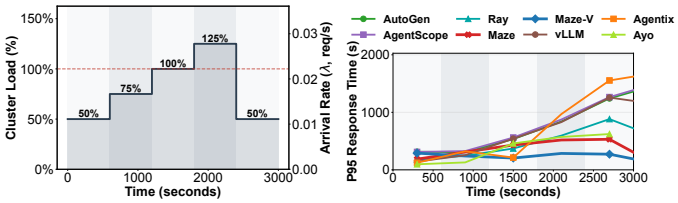
**Metrics.** We evaluate Maze using three primary metrics: *1) System Throughput*, the number of successfully completed workflows per minute (DAGs/Min); *2) DAG Completion Time (DCT)*, the elapsed duration from the arrival of a workflow's first task at the scheduler to the completion of its final task; *3) P95 Response Time*, the 95th percentile of the end-to-end duration from query submission to result return, serving as a key indicator for tail-latency mitigation [29].

**Framework Baselines.** We compare Maze against three categories of distributed paradigms: *1) Agent-level*: AutoGen [7] facilitates complex LLM workflows via multi-agent conversations and supports gRPC-based deployment; AgentScope [21] is an open-source multi-agent platform that implements distributed agent execution through gRPC communication. *2) Model-level*: vLLM [8] is a high-performance inference serving system used here for distributed model-level deployment; Agentix [9] is a program-aware system built on vLLM that schedules inference request applications to mitigate head-of-line blocking. *3) Task-level*: Ray [10] provides a general-purpose decentralized scheduler for executing DAGs; Ayo [11] is an orchestration framework that decomposes workflows into fine-grained primitives for parallel execution.

**Scheduling Algorithm Baselines.** To evaluate HACS, we compare several schedulers within Maze: *1) FCFS* [30], which executes tasks strictly in their order of arrival; *2) PEFT* [31], which uses an Optimistic Cost Table (OCT) for prioritization and processor selection with look-ahead capability; and *3) ATLAS* [9], a dynamic policy prioritizing tasks based on cumulative attained service to estimate critical paths via a Least-Attained-Service (LAS) discipline.

TABLE IV: Quality sanity check across frameworks

| Benchmark (Metric) | Maze | AgentScope | AutoGen |
|---|---|---|---|
| GAIA (Accuracy, %) | 46.51 | 44.18 | 46.51 |
| OpenAGI (BERTScore) | 0.2234 | 0.2406 | 0.2317 |
| $\tau$-bench (Completion Acc., %) | 86.48 | 87.42 | 84.59 |




(a) Cluster Load (b) P95 Response Time

Fig. 6: Performance comparison between frameworks

**Our Method.** We evaluate Maze in two configurations: *1) Maze*, the baseline task-level framework with default backends; *2) Maze-V*, which integrates vLLM services for inference acceleration. We do not perform an exhaustive search and the main configurations are as follows: LLMs are set with temperature 0.6 and 1,024 max tokens; HACS employs $\alpha = 2$ and $\beta = 5$ as the default scheduling parameters; MaLearn initializes with 100 trees once 10 samples are collected, with incremental updates of 50 trees per 10 new samples; Maze-V triggers scaling when request load exceeds 5; MaWorker maintains a pre-warmed pool of 2 GPU workers and 20 CPU workers per node to enable Zero-VRAM standby.

**Configuration and Quality Sanity.** Our evaluation targets agentic deployment, scheduling, and runtime efficiency rather than answer optimization. All frameworks use identical workflow inputs, models, prompts, and decoding settings, and experiments are repeated under the same configurations. As shown in Table IV, Maze maintains comparable workflow quality to AgentScope and AutoGen across all three benchmarks, indicating that the reported system-level improvements are not achieved by sacrificing answer quality.

### B. Framework Performance

To simulate real-world online services, we adopt experimental paradigms from pioneering works [32]–[34] and evaluate performance under dynamic agentic workflow arrivals. As shown in Figure 6a, the experiment defines five load stages: 50%, 75%, 100%, 125%, and a return to 50%. We define 100% load as the theoretical saturation point where the workflow arrival rate matches the cluster's peak aggregate throughput under full resource utilization. Agentic requests arrive following a Poisson distribution, with each stage lasting 10 minutes to ensure the system reaches a steady state.

Experimental results reveal performance disparities under dynamic agentic workflow loads. For P95 response time (Figure 6b), AutoGen, AgentScope, vLLM, and Agentix suffer latency spikes exceeding 1200s at 125% load, while Ayo and Ray reach 622.6s and 882.4s, respectively. Conversely, Maze and Maze-V limit P95 growth to 531s and 285s, with rapid

TABLE V: Resource Utilization and Inter-node Load Imbalance under Saturated Agentic Workflow Load

| Framework | CPU Util. (%) (Mean $\pm$ $\sigma$) | GPU Util. (%) (Mean $\pm$ $\sigma$) |
|---|---|---|
| Ray | 7.3 $\pm$ 2.9 | 27.6 $\pm$ 32.4 |
| Ayo | 6.7 $\pm$ 3.3 | 30.9 $\pm$ 41.3 |
| AutoGen | 6.9 $\pm$ 2.8 | 18.9 $\pm$ 44.7 |
| AgentScope | 6.5 $\pm$ 2.9 | 15.2 $\pm$ 41.9 |
| vLLM | 4.0 $\pm$ 1.6 | 11.8 $\pm$ 33.1 |
| Agentix | 3.6 $\pm$ 1.5 | 12.1 $\pm$ 33.9 |
| **Maze** | **10.0 $\pm$ 1.3** | **46.4 $\pm$ 18.3** |
| Maze-V | **11.5 $\pm$ 1.2** | **35.0 $\pm$ 16.3** |

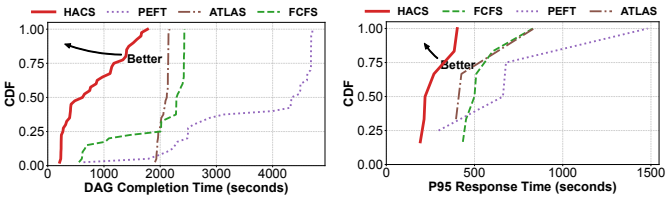
recovery. Regarding resources (Table V), agent- and model-level baselines show low GPU utilization (<19%) with high variance ($\sigma > 33$). Though Ayo and Ray improve throughput, they still exhibit significant imbalance, with Ayo's GPU variance reaching 41.3. Maze achieves the highest GPU utilization (46.4%) with the most stable distribution ($\sigma = 18.3$). These bottlenecks stem from an abstraction mismatch. Agent-level frameworks rely on static binding, resulting in resource fragmentation within the cluster. Model-level systems, being inference-centric, remain oblivious to heterogeneous workflow dependencies and stall on non-LLM components. Crucially, while Ray and Ayo adopt task-level execution, they lack the fine-grained resource abstraction and specialized scheduling awareness required for agentic workloads. Maze addresses these issues by bridging the gap between agentic logic and fine-grained task deployment, which facilitates optimal task placement and balanced saturation across the cluster.

### C. Scheduling Algorithm Comparison

To validate the effectiveness of the HACS algorithm, we compare it with representative scheduling algorithms—FCFS, PEFT, and ATLAS—under both batched and continuous arrival scenarios. HACS consistently demonstrates optimal performance across different arrival patterns. In the batched workload (Figure 7a), HACS exhibits the highest system throughput, followed by ATLAS and FCFS, while PEFT performs the worst. Similarly, under a continuous 85% system load (Figure 7b), HACS maintains the lowest P95 response time for the majority of requests, whereas FCFS and ATLAS show significant latency increases, and PEFT exhibits the most severe long-tail distribution. Effective scheduling for agentic workloads requires capturing their unique properties. The failure of PEFT stems from its heavy reliance on precise a priori knowledge, which is unobtainable in an agentic workflow environment. While ATLAS mitigates starvation via aging, it overlooks critical characteristics such as the heterogeneity and inference intensity within the agentic workload. Ultimately, HACS achieves superior performance by evaluating workflow value through agentic workload characteristics while effectively preventing starvation.

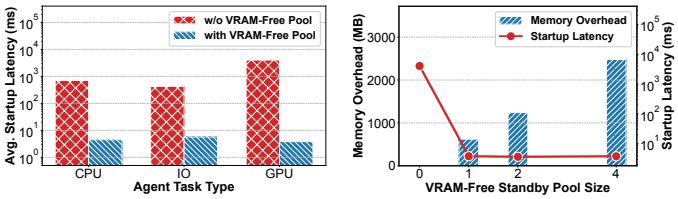
### D. Ablation Study

We evaluate five architectural components of Maze: Resource Anchoring (R.A.), the Scheduling Algorithm (S.A.),




(a) Batched arrivals (b) Continuous arrivals

Fig. 7: Performance comparison of scheduling algorithms under different arrival patterns




(a) Startup latency by task types (b) Cost-benefit of pool size

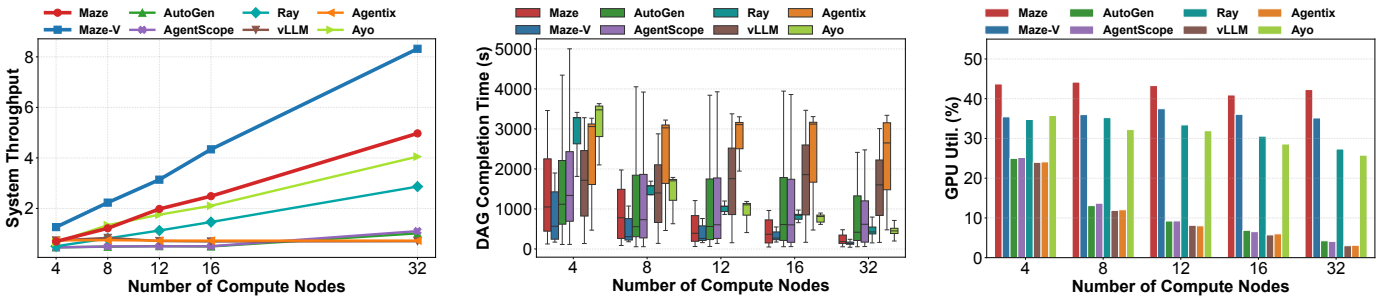
Fig. 8: Performance impact of the VRAM-Free Standby Pool

Heterogeneous Multi-Queues (H.Q.), Task Execution Time Prediction (T.P.), and the VRAM-free Standby Pool (V.S.). We exclude Workflow Context because it is required for workflow execution. We randomly sample 40 agentic requests and submit them simultaneously as a batch. As shown in Table VI, S.A. and R.A. contribute the largest performance gains: removing them increases workflow completion time by 178.6% and 55.9%, respectively. Disabling H.Q. increases completion time by 28.3%, while removing T.P. or V.S. causes smaller but consistent degradation in completion time and P95 latency. These results show that HACS and resource anchoring provide the primary gains, while heterogeneous queues, runtime prediction, and fast worker startup provide complementary improvements.

TABLE VI: Performance overview of ablation study

| Framework | DAG Compl. Time (s) | P95 Resp. Time (s) | CPU Util. (%) | GPU Util. (%) |
|---|---|---|---|---|
| Maze | 672.29 | 1522.83 | 11.25 | 48.25 |
| Maze (w/o R.A.) | 1048.25 | 2062.39 | 6.80 | 30.90 |
| Maze (w/o S.A.) | 1873.26 | 2362.58 | 7.80 | 35.90 |
| Maze (w/o H.Q.) | 862.32 | 1835.25 | 9.12 | 40.71 |
| Maze (w/o T.P.) | 746.62 | 1684.83 | 11.02 | 47.23 |
| Maze (w/o V.S.) | 728.86 | 1603.69 | 10.95 | 46.23 |

The results also explain the source of these gains. Without R.A., task-level execution falls back to coarse resource allocation, increasing fragmentation; without H.Q., GPU-blocked tasks can induce cross-resource head-of-line blocking. T.P. supplies temporal signals for HACS, while V.S. reduces task startup overhead. As shown in Figure 8a, standard worker initialization incurs 4037 ms of startup latency due to heavy library loading, whereas VRAM-free standby reduces it to 3.83 ms. CPU and I/O task startup latencies are similarly reduced to approximately 5 ms. The cost-benefit analysis




(a) System throughput (b) DAG completion time (c) GPU utilization

Fig. 9: Scalability comparison of Maze and baseline frameworks

(Figure 8b) shows an approximately 620 MB host-memory overhead per standby worker without occupying VRAM.

### *E. Scalability*

To evaluate the scalability of different frameworks, we conduct a strong-scaling experiment with a fixed batch workload on clusters ranging from 4 to 32 compute nodes. As shown in Figure 9a and Figure 9b, only task-level frameworks exhibit the capability to scale performance as resources increase. Maze and Maze-V demonstrate near-linear scalability in system throughput; specifically, Maze-V scales from 1.26 to 8.32 workflows/min, significantly outperforming all baselines. While Ray and Ayo also show throughput growth, a sharp divergence appears in response latency. At 32 nodes, Maze reduces the average workflow completion time to 240.4s, whereas Ayo and Ray remain significantly higher at 459.8s and 476.7s, respectively. In contrast, agent-level (AutoGen, AgentScope) and model-level (vLLM, Agentix) frameworks plateau early, with throughput stagnating below 1.1 workflows/min regardless of cluster size.

Ultimately, Maze achieves near-linear throughput scaling over the evaluated 4–32-node range. These results suggest that the performance of Maze is enabled by the fine-grained task-level deployment, agentic workload-aware scheduling, and a stable, scalable runtime. As shown in Figure 9c, the GPU utilization of agent-level frameworks drops sharply as the cluster grows (e.g., from approximately 25% to 4%), likely due to load imbalance under coarse-grained agent placement. Model-level systems similarly fail to translate additional GPU resources into proportional throughput gains because they remain unaware of non-inference stages in agent workflows. In contrast, task-level frameworks maintain more effective GPU utilization as the cluster scales; among them, Maze achieves the strongest throughput scaling by combining fine-grained resource anchoring with agentic workload-aware scheduling.

### *F. Dynamic and Incremental Workflows*

We evaluate Maze under dynamic workflows using the workloads from §V-A, where task dependencies unfold at runtime and the complete DAG is unavailable to the scheduler. We compare against all applicable baselines except Ayo, which requires static workflows. As shown in Table VII, Maze achieves the lowest P95 response time (405.27 s) and the highest GPU utilization (42.82%), reducing P95 latency by 28.1% over Ray, while agent- and model-level baselines remain above 720 s. These results show that HACS can exploit the currently visible subgraph to make effective priority decisions even without complete workflow topology.

TABLE VII: Performance and Resource Utilization Summary under Dynamic workflows

| Framework | P95 Resp. Time (s) | CPU Util. (%) | GPU Util. (%) |
|---|---|---|---|
| Maze | 405.27 | 10.29 | 42.82 |
| AutoGen | 783.73 | 6.67 | 16.22 |
| AgentScope | 765.23 | 6.38 | 15.55 |
| vLLM | 722.87 | 4.19 | 11.64 |
| Agentix | 804.35 | 3.82 | 12.42 |
| Ray | 563.28 | 7.27 | 25.45 |

### *G. Portability to Ascend NPUs*

To evaluate Maze beyond the NVIDIA platform, we further deploy it on an 8-NPU Ascend server. We use the 40-query batched workload setting as in §V-D and select representative agent-level (AutoGen), model-level (vLLM), and task-level (Ray) baselines. As shown in Table VIII(a), Maze achieves the lowest P95 response time (687.76 s) and the highest NPU utilization (36.05%). Compared with the task-level baseline Ray, Maze reduces P95 latency while improving NPU utilization, demonstrating that its task-level abstraction and scheduling benefits extend beyond NVIDIA GPUs.

### *H. Task-Level Control-Plane Overhead*

Moving from agent-level to task-level execution introduces additional per-task control costs. Using traces from the dynamic-load experiment, Table VIII(b) reports the overhead of MaLearn prediction, MaPath scheduling, and communication/coordination. The total control-plane overhead is only 43.1 ms per task.

### *I. Scope and Limitations*

Maze primarily targets heterogeneous, dependency-rich agent workflows, where task-level execution can exploit cross-resource parallelism and workflow structure. Its gains can be smaller for short or homogeneous workflows, which provide fewer opportunities for heterogeneous-resource scheduling and critical-path prioritization. For incremental workflows,

TABLE VIII: Portability and task-level control-plane overhead

**(a) Portability on Ascend NPUs**

| Framework | P95 Resp. (s) | CPU Util. (%) | NPU Util. (%) |
|---|---|---|---|
| Maze | 687.76 | 11.88 | 36.05 |
| AutoGen | 908.18 | 6.94 | 15.83 |
| vLLM | 1115.52 | 4.76 | 10.32 |
| Ray | 786.44 | 8.12 | 23.45 |

**(b) Per-task control-plane overhead**

| Component | Overhead |
|---|---|
| MaLearn prediction | 17.4 ms |
| MaPath scheduling | 4.3 ms |
| Communication/coordination | 21.4 ms |
| **Total** | **43.1 ms** |

scheduling decisions are limited by the currently visible subgraph; therefore, benefits may decrease when only a small subgraph is visible at a time. Beyond workload characteristics, our evaluation covers strong scaling on NVIDIA A100 GPUs and portability to Ascend NPUs, but does not include weak scaling or broader accelerator architectures. We also do not evaluate model-sharded inference; Maze can integrate sharded inference backends as pluggable model services, while optimizing model sharding itself is outside the scope of this work.

## VI. Related Work

**Distributed Inference Serving Systems.** Distributed inference serving systems face an inherent throughput–latency trade-off and optimize resource usage through batching and memory management. Existing work spans two layers. At the engine layer, systems such as DeepSpeed-Inference [35], FlexGen [36], vLLM [8], and InfiniGen [37] improve single-model efficiency via optimized kernels, memory offloading, and dynamic KV-cache management. At the serving layer, recent systems [38]–[40] and Agentix [9] introduce various scheduling techniques to improve multi-tenant performance, with Agentix specifically addressing head-of-line blocking between long- and short-running requests. However, these systems remain inference-centric and operate at the request level, overlooking heterogeneous, multi-stage agent workflows and thus falling short of meeting end-to-end agent SLOs.

**Distributed Frameworks for LLM Agents.** While many LLM agent frameworks [6], [7], [21], [41], [42] have been developed, there are a very limited number of frameworks that support distributed deployment of LLM agents. AutoGen [7] employs a host-worker architecture via gRPC, where worker nodes maintain persistent connections to a host that acts as a message broker, routing requests to individual agent instances. AgentScope [21] adopts the Actor model, enabling developers to register agent definitions and instantiate them as remote actors on compute nodes through proxy objects. LangGraph [41] represents agent logic as graph-structured workflows and, together with LangServe [42], supports serving them as distributed agent applications. Despite these capabilities, their native execution models remain fundamentally agent-centric. Once an agent is instantiated, its entire workflow is statically bound to the resources of a single node. This coarse-grained agent-level binding prevents the runtime from decoupling heterogeneous tasks, inevitably leading to resource fragmentation and load imbalance across the cluster. Alternatively, Ayo [11] focuses on enhancing end-to-end parallelism by representing workflows as fine-grained primitives. However, Ayo primarily addresses orchestration patterns and fails to bridge the significant gap between high-level agentic logic and task-level execution. Furthermore, it does not incorporate agent-specific workload characteristics into its scheduling logic, nor does it provide a robust runtime capable of supporting large-scale distributed environments.

**Cluster Scheduling.** Cluster scheduling strategies comprise static and dynamic approaches. Static heuristics, such as HEFT [43] and PEFT [31], optimize workflow makespan using global DAG knowledge but require precise task durations and complete topology, making them unsuitable for agentic workflows with input-dependent execution and dynamic structure. Dynamic frameworks like Ray [10] enable fine-grained parallelism, achieving balanced load, resource packing, and data locality, yet treat tasks as independent units, lacking awareness of latency-critical subtasks. Policies such as ATLAS in Agentix [9], along with other serving-oriented schedulers [44]–[46], primarily target homogeneous inference workloads and therefore overlook task heterogeneity, DAG structure, and latency-critical subtasks in agent workflows.

## VII. Conclusion

There is a path for every agent. In this paper, we design Maze, a framework to enable efficient and scalable distributed LLM agent deployment via fine-grained task-level execution. Maze adopts a modular design and provides easy-to-use Python APIs. Our extensive experiments show that Maze significantly outperforms the other LLM agent frameworks, reducing the P95 response time by up to 54.2%, increasing throughput by up to 2.9×, and improving GPU utilization by 50%. We believe Maze offers a promising foundation for future work on serving increasingly complex, dynamic, and multi-tenant agentic workloads.

## VIII. Acknowledgments

The authors used Gemini 3 [47] to assist in polishing the submission text, primarily for grammar checking and enhancing the overall readability of the paper. The authors have reviewed and verified the accuracy of the resulting content and take full responsibility for the paper's contents. This work is supported by New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0122103), NSFC-RGC under Grant 6246116033, and National Natural Science Foundation of China (Grant No. U25B2023).

## REFERENCES


[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024. [Online]. Available: https://doi.org/10.1007/s11704-024-40231-1

[2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Qin, Y. Zheng, X. Qiu, X. Huang, Q. Zhang, and T. Gui, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025. [Online]. Available: https://doi.org/10.1007/s11432-024-4222-0

[3] G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=fibxvahvs3

[4] S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan, "$\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains," in *Proceedings of the Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=roNSXZpUDN

[5] Y. Ge, W. Hua, K. Mei, J. Ji, J. Tan, S. Xu, Z. Li, and Y. Zhang, "OpenAGI: When LLM meets domain experts," in *Proceedings of the 37th Conference on Neural Information Processing Systems*, vol. 36, 2023, pp. 5539–5568. [Online]. Available: https://doi.org/10.52202/075280-0242

[6] H. Chase, "LangChain," https://github.com/langchain-ai/langchain/tree/langchain==0.3.27, 2022, accessed: 2025-08-21.

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proceedings of the First Conference on Language Modeling*, 2024. [Online]. Available: https://openreview.net/forum?id=BAakY1hNKS

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[9] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, "Agentix: An efficient serving engine for LLM agents as general programs," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026, pp. 2443–2459. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/luo

[10] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 2018, pp. 561–577. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/moritz

[11] X. Tan, Y. Jiang, Y. Yang, and H. Xu, "Towards end-to-end optimization of LLM-based applications with ayo," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1302–1316. [Online]. Available: https://doi.org/10.1145/3676641.3716278

[12] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, Dec. 2022, art. no. 138. [Online]. Available: https://doi.org/10.1145/3539606

[13] Z. Li, S. Xu, K. Mei, W. Hua, B. Rama, O. Raheja, H. Wang, H. Zhu, and Y. Zhang, "AutoFlow: Automated workflow generation for large language model agents," *arXiv preprint arXiv:2407.12821*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.12821

[14] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, vol. 37, 2024, pp. 50 528–50 652. [Online]. Available: https://doi.org/10.52202/079017-1601

[15] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. V. A, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into state-of-the-art pipelines," in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=sY5N0zY5Od

[16] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of llm agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024. [Online]. Available: https://arxiv.org/abs/2402.02716

[17] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 929–945. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/lin-chaofan

[18] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=oKn9c6ytLx

[19] H. Kang, Q. Zhang, H. Cai, W. Xu, T. Krishna, Y. Du, and T. Weissman, "Win fast or lose slow: Balancing speed and accuracy in latency-sensitive decisions of LLMs," in *Proceedings of the 39th Conference on Neural Information Processing Systems*, vol. 38, 2025, pp. 167 482–167 504. [Online]. Available: https://doi.org/10.52202/085713-5044

[20] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," in *Proceedings of the Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=chfJJYC3iL

[21] D. Gao, Z. Li, X. Pan, W. Kuang, Z. Ma, B. Qian, F. Wei, W. Zhang, Y. Xie, D. Chen, L. Yao, H. Peng, Z. Zhang, L. Zhu, C. Cheng, H. Shi, Y. Li, B. Ding, and J. Zhou, "AgentScope: A flexible yet robust multi-agent platform," *arXiv preprint arXiv:2402.14034*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.14034

[22] CrewAI Inc., "CrewAI: A framework for orchestrating autonomous AI agents," https://github.com/crewAIInc/crewAI/tree/0.165.1, 2025, version 0.165.1; released 2025-08-19; accessed 2025-08-21.

[23] Phidata, "Phidata," 2025, version 2.7.10; accessed 2025-08-21. [Online]. Available: https://github.com/agno-agi/phidata/tree/v2.7.10

[24] Pydantic, "Pydantic AI: A python agent framework for generative AI," https://github.com/pydantic/pydantic-ai/tree/v0.7.4, 2025, version 0.7.4; released 2025-08-20; accessed 2025-08-21.

[25] Z. Asgar, M. Nguyen, and S. Katti, "Efficient and scalable agentic ai with heterogeneous systems," *arXiv preprint arXiv:2507.19635*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.19635

[26] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017)*, ser. Advances in Neural Information Processing Systems, vol. 30. Curran Associates, Inc., 2017, pp. 3146–3154. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[28] J. A. van Mieghem, "Dynamic scheduling with convex delay costs: The generalized c$\mu$ rule," *The Annals of Applied Probability*, vol. 5, no. 3, pp. 809–833, 1995. [Online]. Available: https://doi.org/10.1214/aoap/1177004706

[29] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. [Online]. Available: https://doi.org/10.1145/2408776.2408794

[30] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 7th ed. Hoboken, NJ: John Wiley & Sons, 2005. [Online]. Available: https://openlibrary.org/books/OL21187520M

[31] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 682–694, 2014. [Online]. Available: https://doi.org/10.1109/TPDS.2013.57

[32] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proceedings of the ACM Special Interest Group on Data Communication*, 2019, pp. 270–288. [Online]. Available: https://doi.org/10.1145/3341302.3342080

[33] Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou, "Aegaeon: Effective GPU pooling for concurrent LLM serving on the market," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, ser. SOSP '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1030–1045. [Online]. Available: https://doi.org/10.1145/3731569.3764815

[34] Z. Lin, C. Li, L. Tian, and B. Zhang, "A scheduling algorithm based on reinforcement learning for heterogeneous environments," *Applied Soft Computing*, vol. 130, p. 109707, 2022. [Online]. Available: https://doi.org/10.1016/j.asoc.2022.109707

[35] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proceedings of SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15. [Online]. Available: https://doi.org/10.1109/SC41404.2022.00051

[36] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 094–31 116. [Online]. Available: https://proceedings.mlr.press/v202/sheng23a.html

[37] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 155–172. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/lee

[38] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, "ServerlessLLM: Low-Latency serverless inference for large language models," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 135–153. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/fu

[39] I. Gim, Z. Ma, S.-s. Lee, and L. Zhong, "Pie: A programmable serving system for emerging LLM applications," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, ser. SOSP '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 415–430. [Online]. Available: https://doi.org/10.1145/3731569.3764814

[40] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 117–134. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/agrawal

[41] LangChain, Inc., "LangGraph," https://github.com/langchain-ai/langgraph/tree/1.2.2, 2024, accessed: May 2026.

[42] ——, "LangServe," https://github.com/langchain-ai/langserve/tree/v0.3.3, 2023, accessed: May 2026.

[43] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE transactions on parallel and distributed systems*, vol. 13, no. 3, pp. 260–274, 2002. [Online]. Available: https://doi.org/10.1109/71.993206

[44] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, "Fairness in serving large language models," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 965–988. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/sheng

[45] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 173–191. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/sun-biao

[46] Y. Fu, S. Zhu, R. Su, A. Qiao, I. Stoica, and H. Zhang, "Efficient LLM scheduling by learning to rank," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, vol. 37, 2024, pp. 59 006–59 029. [Online]. Available: https://doi.org/10.52202/079017-1882

[47] S. Pichai, D. Hassabis, and K. Kavukcuoglu, "A new era of intelligence with Gemini 3," Google, November 2025, accessed: 2026-08-11. [Online]. Available: https://blog.google/products/gemini/gemini-3/